Continuous Delivery & Continuous Deployment

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.

Learning Objectives

- Distinguish between Continuous Delivery (CD) and Continuous Deployment (CD) and explain why the distinction matters in cloud environments
- Describe the stages of a deployment pipeline from build artifact to production
- Identify common deployment strategies (rolling, blue-green, canary) and their trade-offs
- Map CD concepts to specific AWS and GCP services
- Explain the role of Infrastructure as Code and immutable infrastructure in enabling CD

From CI to CD — Closing the Gap

- Quick recap: CI ends with a tested, versioned **build artifact** (AMI, Docker image, JAR, etc.)
- The problem CI alone does not solve: how does that artifact reach production safely?
- Vocabulary clarification:
  - **Continuous Delivery** — every commit *can* be deployed to production at the push of a button (human gate)
  - **Continuous Deployment** — every commit that passes the pipeline *is* automatically deployed to production (no human gate)
- Industry reality: most organizations practice Continuous **Delivery**; fully automated deployment to production is less common and requires high confidence in test coverage and observability

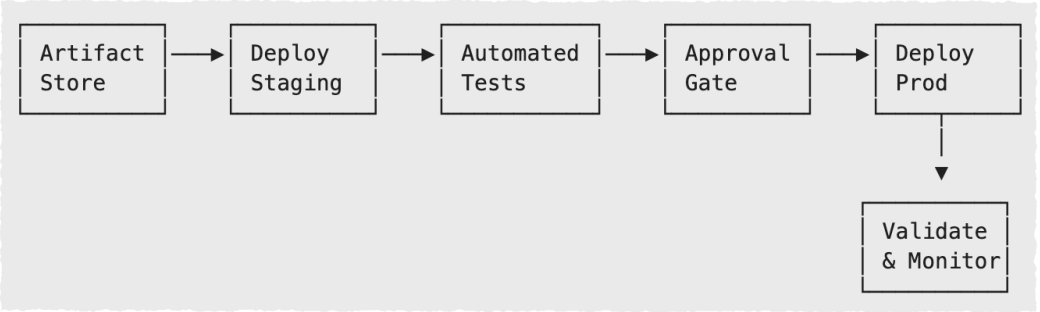
```
CI already covered
        |
        ▼
┌──────────────┐
│ Build Artifact │  (AMI, Docker image, ZIP, etc.)
└──────────────┘
        |
        ▼
┌──────────────────────────────────────────────┐
│            Deployment Pipeline               │
│                                              │
│  Staging ──▶ Integration Tests ──▶ Approval? │
│                                   |     |    |
│                             (human gate) (auto)│
│                                   |     |    |
│                                   ▼     ▼    |
│                              Production      |
└──────────────────────────────────────────────┘
        Continuous        Continuous
        Delivery          Deployment
      (button click)    (fully automatic)
```

Anatomy of a Deployment Pipeline

Stages after CI completes:

- **Artifact storage** — store the immutable artifact (e.g., container registry, S3 bucket, AMI registry)
- **Deploy to staging/pre-production** — identical (or near-identical) environment to production
- **Automated acceptance / integration / smoke tests** — validate the artifact behaves correctly in a production-like setting
- **Approval gate** (Continuous Delivery) or auto-promote (Continuous Deployment)
- **Deploy to production**
- **Post-deployment validation** — health checks, synthetic monitoring, log/metrics verification
- **Rollback path** — what happens when something goes wrong

- Key principle: **the artifact that was tested is the artifact that gets deployed** — never rebuild for production
- Immutable artifacts eliminate "works on my machine" at the infrastructure level



Deployment Strategies in the Cloud

In-Place (Rolling) Deployment

- Replace instances one at a time (or in batches) behind a load balancer
- During deployment, both old and new versions serve traffic simultaneously
- Risk: if the new version is broken, some users are affected before rollback completes
- AWS example: CodeDeploy with EC2 Auto Scaling (OneAtATime, HalfAtATime, AllAtOnce)
- GCP example: Compute Engine managed instance group rolling update

```
Load Balancer
    │
    ├──▶ Instance A  [v1] ──▶ [v2] ✓  (updated first)
    ├──▶ Instance B  [v1] ──▶ [v2] ✓  (updated second)
    └──▶ Instance C  [v1] ──▶ ...     (updating now)
```

Blue-Green Deployment

- Two identical environments: **Blue** (current production) and **Green** (new version)
- Deploy new version to Green, run tests against Green, then switch traffic (DNS or load balancer swap)
- Rollback = switch traffic back to Blue
- Cost: you pay for two full environments during the switchover window
- AWS example: Elastic Beanstalk URL swap, Route 53 weighted routing, ALB target group switching
- GCP example: Cloud Run revisions with traffic splitting, GCE instance groups behind a load balancer

```
┌──────────────────┐
│ Load Balancer    │
│ / DNS Switch     │
└──────────────────┘
        │
    ┌───┴───┐
    ▼       ▼
┌─────────┐ ┌─────────┐
│ BLUE    │ │ GREEN   │
│ (v1)    │ │ (v2)    │
│ current │ │ new     │
└─────────┘ └─────────┘

Step 1: Deploy v2 to Green
Step 2: Test Green
Step 3: Switch traffic Blue → Green
Step 4: Green becomes the new "current"
```

Canary Deployment

- Route a small percentage of traffic (e.g., 5%) to the new version
- Monitor error rates, latency, business metrics
- Gradually increase traffic if metrics are healthy; roll back instantly if not
- Most sophisticated strategy — requires good observability
- AWS example: CodeDeploy with Lambda/ECS canary configuration, ALB weighted target groups
- GCP example: Cloud Run traffic splitting (e.g., 95/5), GKE with Istio traffic management

```
                ┌──────────────────┐
                │  Load Balancer   │
                └──────────────────┘
                        │
                ┌───────┴────────┐
            95% ▼            ▼ 5%
        ┌──────────┐    ┌──────────┐
        │  Stable  │    │  Canary  │
        │  (v1)    │    │  (v2)    │
        └──────────┘    └──────────┘

If canary healthy → shift to 25% → 50% → 100%
If canary unhealthy → route 100% back to v1
```

Strategy Comparison

Criteria	Rolling	Blue-Green	Canary
Downtime risk	Low	Very low	Very low
Rollback speed	Moderate	Fast (traffic switch)	Fast (traffic shift)
Infrastructure cost	Lowest	High (2× environments)	Moderate
Complexity	Low	Moderate	High
Mixed versions during deploy?	Yes	Brief or none	Yes (controlled)
Best for	Simple apps, cost-sensitive	Mission-critical, fast rollback needed	Large user bases, gradual risk reduction

What Makes CD Possible — Key Enablers

- **Immutable infrastructure** — never patch a running server; replace it with a new one built from the artifact
  - Eliminates configuration drift
  - Makes rollback trivial: just point back to the previous artifact
- **Infrastructure as Code (IaC)** — environments defined in version-controlled templates, so staging truly mirrors production
- **Feature flags / feature toggles** — decouple *deployment* (putting code on servers) from *release* (exposing features to users)
  - Deploy dark: code is in production but behind a flag
  - Enables Continuous Deployment without exposing unfinished features
- **Comprehensive automated testing** — CD only works if you trust your test suite
- **Observability** — logs, metrics, traces, and alerts must be in place to catch regressions post-deploy

Common Pitfalls and Real-World Considerations

- Database migrations are the hardest part of CD — schema changes must be backward-compatible
- Avoid "big bang" deployments: if you deploy once a month, each deploy carries a month of risk
- Secrets management: deployment pipelines need access to credentials — use AWS Secrets Manager, GCP Secret Manager, or HashiCorp Vault — never hardcode
- Cost awareness: blue-green doubles your infrastructure bill during transitions; canary requires investment in observability tooling

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS